

Análise para o App Flutter

Navix Route Intelligence — Riscos de Retrabalho no Cliente Móvel

Revisor: Principal Architect / CTO · Data: 2026-07-12 · Escopo: contratos, endpoints, auth e features (Tracking, Import, POD, Optimizer) · Modo: somente leitura — nenhum código alterado

Contexto

O app Flutter **já existe** e é substancial (apps/mobile: auth, dashboards, imports, optimizer, POD com fila offline, tracking). Portanto os pontos abaixo são **retrabalho real** — coisas já codadas (ou a codar) no cliente que precisarão mudar quando o backend evoluir, ou que forçam gambiarras no app. Cada item está aterrado em arquivos concretos do repositório.

CRÍTICO — retrabalho estrutural

1. Acoplamento oculto Cookie ↔ X-Auth-Mode: bearer (Auth / Refresh Token)

A migração recente colocou o refresh token em **cookie HttpOnly** para a web. O backend só devolve o refresh token no corpo quando a requisição envia X-Auth-Mode: bearer (auth-cookie.ts, auth.controller.ts). Hoje o app depende de um header **global** no Dio (dio_client.dart:26).

Risco: qualquer novo Dio (upload, download isolado) que não replicar o header recebe login/refresh **sem** refresh token → sessão quebra em silêncio. O Set-Cookie ainda vem em toda resposta e é ignorado (mobile não tem cookie jar) — comportamento implícito e frágil. **Ação:** contrato explícito de cliente nativo (bearer por padrão/rota dedicada) ou invariante testada.

2. tenantId (UUID) obrigatório no login (Multi-Tenant)

login.dto.ts exige tenantId como @IsUUID(). Um motorista **não conhece o UUID do tenant**. O app terá que inventar um fluxo (deep link, QR, seleção) e, quando o backend resolver por **e-mail** → **tenant** ou subdomínio (o contrato admite evoluir), a tela de login e o AuthRepository serão refeitos. **Ação:** resolver identidade do tenant por e-mail/handle antes de o app crescer.

3. Sem Idempotency-Key (Offline First / POD / Tracking)

O header está no CORS (main.ts) mas **não há implementação**. Para um app offline-first que re-sincroniza fila, isso é grave: **POD** duplicado após reconexão retorna **409** (submit-pod.use-case.ts:44) — o pod_sync_cubit precisa tratar 409 como sucesso idempotente; **Tracking** não deduplica replay de posições. **Ação:** adotar Idempotency-Key no backend; até lá, tratamento ad-hoc de duplicatas no cliente (que muda de novo quando o header existir).

4. Otimização síncrona vs. timeout do Dio (Route Optimizer)

POST /route-plans e /route-plans/mine respondem 201 com status: 'completed' — **sem job/202**. O Dio tem receiveTimeout de **20s** (dio_client.dart:24). Rota com muitas paradas pode estourar → timeout sem recurso para acompanhar. Quando o backend migrar para fila/202 + jobId (ADR-0007), **todo o optimizer_repository/cubit será reescrito** (de request-resposta para polling/push).

ALTO

5. Sem transporte de tempo real (Tracking)

Não há WebSocket/SSE — só REST. O tracking de frota depende de **polling** (GET tracking/positions/latest, tracking/me/latest). Ao introduzir push (Fase 2), a camada de dados do fleet_tracking_cubit muda.

6. Ingestão de posição sem batch (Tracking / Offline)

update-position.dto.ts aceita **uma** posição por POST (recordedAt opcional, bom para offline). Ao reconectar, o app faz **N requests** para N posições acumuladas. Um POST /tracking/positions:batch mudaria o location_sharing_cubit/tracking_repository.

7. Mídia do POD em base64 no corpo JSON (Proof of Delivery / Offline)

photo/signature são string (data URL) no corpo, limite de **8 MB** (main.ts). A fila offline (pod_queue_store) guarda base64 — infla ~33%, pesa em memória/disco e em rede móvel; sem multipart/upload resumável. Migrar para object storage + URL assinada (dívida A4 da auditoria) **reescreve** captura, fila e envio do POD.

8. Sem sync incremental / delta (Offline First — transversal)

As listagens são só paginação **offset** (page/pageSize) sem filtro updatedSince e sem endpoint de changes/delta. Cache offline fará full-refetch; scroll infinito com dados vivos duplica/salta → provável migração para **cursor**. Ambos alteram a camada de repositório/cache do app.

MÉDIO

9. details de validação nunca são enviados (Consistência / DTOs)

O contrato define ApiErrorDetail[] (erro por campo), mas o filtro **não popula** details (domain-exception.filter.ts) — o ValidationPipe vira uma message string. Formulários (login, POD, import) não conseguem destacar o campo com erro; quando details for implementado, a UI de erro muda.

10. Escopo 'me' inconsistente entre módulos (Endpoints)

auth/me, **me/profile**, **me/settings**, tracking/me/latest, route-plans/mine — cinco padrões para 'recurso do usuário atual'. Dificulta um cliente REST genérico; padronizar depois quebra o app. Vale congelar a convenção agora.

11. RBAC duplicado no cliente (RBAC)

As roles (admin|dispatcher|fleet_manager|driver) vêm no JWT, mas **não há endpoint de capabilities**; o app mapeia role→telas manualmente (espelhando roles.ts da web). Cada mudança de permissão exige editar web e mobile. Um /auth/me com capabilities evitaria a duplicação.

12. Links de paginação relativos e offset (DTOs)

buildCollection gera next/prev como '/api/v1/...?page=2' (pagination.ts). O app precisa compor host+prefixo; e por ser offset, o scroll infinito é frágil (ver item 8).

BAIXO — bom, mas vale fixar como contrato

- **Datas:** todas ISO string (createdAt, recordedAt...) — consistente e Flutter-friendly. Congelar como padrão (sempre UTC, sufixo Z).

- **Versionamento:** /api/v1 uniforme em todos os controllers (exceto health, que é infra).
- **Envelope:** { data } / { data, meta, links } / { error: { code, message, requestId } } consistente. O requestId ajuda muito o suporte no app.
- **Import Center:** multipart em POST /imports/preview + confirm, com role driver permitido — adequado ao motorista autônomo. Atenção ao timeout em arquivos grandes (mesmo risco do item 4).

Tabela de priorização (retrabalho)

#	Área	Área de retrabalho	Prioridade
1	Auth / Refresh	Acoplamento cookie↔bearer implícito	Crítico
2	Multi-Tenant	tenantId UUID no login	Crítico
3	Offline / POD / Tracking	Sem Idempotency-Key (409 no re-sync)	Crítico
4	Route Optimizer	Síncrono vs timeout; migração p/ job	Crítico
5	Tracking	Sem WebSocket/SSE (polling)	Alto
6	Tracking / Offline	Sem batch de posições	Alto
7	POD / Offline	Mídia base64 no corpo	Alto
8	Offline First	Sem sync incremental/cursor	Alto
9	APIs / DTOs	details de validação ausentes	Médio
10	Endpoints	Escopo 'me' inconsistente	Médio
11	RBAC	Permissões duplicadas no cliente	Médio
12	DTOs	Paginação relativa + offset	Médio

Recomendação de sequência

Resolver **antes de escalar o app** os itens **1–4** (contrato de auth nativo, login por e-mail→tenant, Idempotency-Key e otimização assíncrona). Cada um deles reescreve uma **camada inteira** do cliente (AuthRepository, login, sync de POD/tracking, optimizer_repository) — quanto mais tarde, mais caro o retrabalho.